

Project Report: Acceleration of the LLM.c

DD2360 – Applied GPU Programming - Group 14
Diogo Paulo, Hugo Dezerto, Maria Carolina Sebastião

February 8, 2026

All members contributed equally to the development of this project.

1 Introduction

Large Language Models (LLMs) have become a central component of modern machine learning applications, including natural language processing, text generation and conversational systems. These models rely heavily on deep transformer architectures, which involve repeated large-scale matrix multiplications, attention mechanisms and non-linear transformations. As model sizes and sequence lengths grow, the computational and memory demands of these operations increase significantly, making efficient execution a critical challenge.

In this project, we focus on accelerating a C/CUDA implementation of GPT-2 provided by the open-source project `llm.c` by Karpathy [1]. The application implements both training and inference for a transformer-based language model and serves as a realistic workload representation of modern deep learning systems.

The dominant computations in this application consist of dense linear algebra operations, in particular matrix multiplications within the attention and feed-forward layers. These operations are executed repeatedly across many layers and time steps, placing heavy demands on the system due to their high arithmetic intensity and huge data movement requirements.

On traditional CPU-based implementations, these quickly become performance bottlenecks, resulting in long execution times. On the other hand, GPUs are well-suited for this type of workload because they offer massive parallelism, high memory bandwidth and specialized hardware units designed for matrix operations.

The project applies two main topics from Group A: the use of NVIDIA-provided libraries and reduced/mixed precision computation. Specifically, custom matrix multiplication kernels are replaced with highly optimized cuBLAS implementations and FP16 mixed precision is explored to reduce memory bandwidth usage and enable Tensor Core acceleration. Through profiling, these techniques are applied to the baseline implementation to identify and address performance bottlenecks. Performance is evaluated by comparing execution time, throughput and numerical correctness before and after optimization, allowing us to assess both the benefits and trade-offs of these GPU-specific optimizations on a real-world transformer-based LLM workload.

2 Experimental Setup

All experiments were conducted on Google Colab using a single NVIDIA Tesla T4 GPU (Turing architecture, 15 360 MiB VRAM) and an Intel Xeon CPU (2 vCPUs, 12 GiB RAM).

The software environment consisted of Ubuntu 22.04.5 LTS, NVIDIA driver 550.54.15 and CUDA Toolkit 12.5 (`nvcc 12.5.82`). Profiling and linear algebra operations used `nvprof 12.5.82`, Nsight Compute (`ncu 2024.2.1.0`) and cuBLAS.

3 Methodology

This project follows a performance engineering workflow: (i) establish a correct baseline, (ii) profile to identify the dominant bottlenecks, (iii) apply targeted optimizations focused on the most expensive operations and (iv) validate numerical behavior and quantify performance impact after each change.

3.1 Baseline Implementation

The baseline is the provided CUDA GPT-2 training implementation (`train_gpt2_fp32.cu`), which loads a GPT-2 124M checkpoint (`gpt2_124M.bin`), initializes cuBLAS and trains for one epoch on the TinyShakespeare dataset using a fixed batch size and sequence length ($B = 4$, $T = 1024$).

Each iteration performs data loading, a forward pass, gradient reset, backward pass and an AdamW parameter update. End-to-end iteration timing includes an explicit `cudaDeviceSynchronize()` call before stopping the CPU timer, and validation loss computation and sample generation (sanity checks) are executed periodically according to the command-line configuration.

All model parameters and activations use FP32 precision. The GPU executes the forward, backward and update kernels, while the CPU handles data loading, logging and coordination of validation and sampling. Some operations require device-to-host transfers, which may introduce additional `cudaMemcpy` and synchronization overhead.

3.2 Profiling and Bottleneck Identification

Our profiling goal was to identify (1) which kernels dominate total execution time and (2) whether the main hotspot is limited by compute throughput, memory bandwidth or occupancy/resource constraints.

We used a two-stage profiling approach:

- **nvprof** for overall performance profiling: kernel time breakdown and CUDA runtime API overheads (e.g., `cudaMemcpy`, `cudaDeviceSynchronize`).
- **Nsight Compute (ncu)** for hotspot microanalysis: per-kernel hardware metrics such as compute vs memory throughput, achieved occupancy, registers per thread and shared memory usage.

First, we run **nvprof** to obtain the dominant GPU kernels and the main CUDA API costs. Then, we select the top kernel and analyze one representative launch with **ncu** to understand utilization and limiting resources.

We focus on: (i) kernel time percentage and call count (from **nvprof**), (ii) time spent in `cudaMemcpy` and synchronization (from **nvprof**), (iii) compute/memory throughput, occupancy, registers/thread and shared memory/block (from **ncu**).

Because Nsight Compute collects performance counters, it introduces profiling overhead; therefore, full performance is reported from normal (non-**ncu**) runs, while **ncu** is used for diagnostic analysis of a small number of kernel launches.

3.3 Optimizations

Based on the profiling results, we applied a set of targeted optimizations to the baseline implementation, focusing on accelerating the most computationally expensive components.

3.3.1 Replacement of Custom GEMMs with cuBLAS

Profiling revealed that matrix multiplications in the linear layers dominate the runtime of the baseline implementation (see Section 4.1). These operations were originally implemented using a hand-written CUDA kernel (`matmul_forward_kernel14`) with fixed tiling and shared-memory blocking.

To improve performance, we replaced this custom kernel with NVIDIA’s cuBLAS library, which provides highly optimized GEMM (General Matrix Multiplications) implementations tuned for each GPU architecture [2]. The linear-layer function (`matmul_forward`) was refactored to optionally route all dense matrix multiplications through cuBLAS, controlled at runtime via the `LLMC_ENABLE_CUBLAS_LINEAR` environment variable, which allows for direct comparison against the baseline kernel.

When enabled, cuBLAS performs FP32 matrix multiplications using `cublasSgemm`. Activations are interpreted as a matrix of shape $(B \cdot T, C)$, where C is the input feature dimension of the linear layer, and

are multiplied with the transposed weight matrix of shape (OC, C) , where OC denotes the corresponding output feature dimension. The `CUBLAS_OP_T` flag is used to accommodate the row-major layout of the codebase without explicit data reordering. Bias terms are applied afterward using a lightweight CUDA kernel, preserving the original computation behavior.

Replacing the custom kernel with cuBLAS is expected to improve performance when matrix multiplications are sufficiently large and dominate runtime. For smaller problem sizes or when additional overheads are significant, the performance gains may be limited.

3.3.2 Reduced and Mixed Precision Computation

To further reduce both inference and training latency while maintaining numerical stability, we adopted a mixed-precision strategy that selectively applies reduced precision only to the most performance-critical operations.

We evaluated the use of both FP16 and BF16 precision formats, but ultimately focused on FP16 because the NVIDIA T4 GPU does not provide BF16 Tensor Core support. As a result, BF16 would not have meaningful performance improvements on this hardware, whereas FP16 enables Tensor Core acceleration [3].

Our implementation follows a conservative mixed-precision design. All numerically sensitive and reduction-heavy components remain in FP32, including the master copy of the model parameters, gradient accumulation, the AdamW optimizer state and all non-matrix-multiplication kernels such as layer normalization, softmax, GELU and residual additions.

Reduced precision is used only where it offers a clear performance benefit with acceptable numerical risk. Specifically, FP16 is applied to selected large matrix multiplications in the forward pass, including the linear layers in the attention projections, the MLP blocks and the final output projection. For these layers, we maintain read-only FP16 shadow copies of the weights on the GPU. Before each such matrix multiplication, the corresponding FP32 activation block is temporarily cast into an FP16 scratch buffer. The GEMMs are then executed using `cublasGemmEx`, which consumes FP16 inputs and weights while accumulating and writing the results back in FP32. This approach ensures that all downstream computations continue in full precision, while still benefiting from the increased throughput and reduced memory bandwidth provided by FP16 Tensor Core execution [4].

We also did not apply mixed precision to the backward pass or loss computation. These already use cuBLAS SGEMM with TF32 math and contribute less to the total runtime than the forward GEMMs. Converting gradients between precisions would also complicate the AdamW optimizer with little expected performance benefit.

4 Results

This section reports baseline correctness, performance and GPU profiling results, which serve as a reference for evaluating the proposed optimizations. Subsequent results compare iteration time, throughput and profiling metrics for both training and inference against this baseline.

4.1 Baseline

Correctness was checked using the provided GPU tests with both `LLMC_ENABLE_CUBLAS_LINEAR` and `LLMC_ENABLE_MIXED_PRECISION` disabled. In this setup, the FP32 version passed all checks on logits, loss and gradients, giving a solid reference for later comparisons.

Training performance is measured using the iteration time reported by the program, with explicit device synchronization at the end of each step using `cudaDeviceSynchronize()`. On an NVIDIA Tesla T4 GPU with $B = 4$ and $T = 1024$ (74 training iterations, including periodic validation and sampling), the baseline achieves an average iteration time of **1063.97 ms/iter** and a training throughput of approximately **3850 tokens/s**.

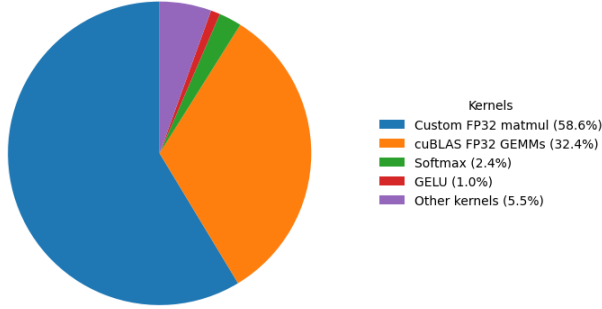


Figure 1: Baseline GPU kernel time breakdown (FP32).

Kernel (baseline)	GPU time share [%]	Total time
<code>matmul_forward_kernel14</code>	58.65	120.27 s
<code>volta_sgemm_128x64_nn</code>	10.32	21.16 s
<code>volta_sgemm_128x64_nt</code>	6.75	13.84 s
<code>volta_sgemm_128x64_tn</code>	5.54	11.35 s
<code>volta_sgemm_64x64_nn</code>	4.98	10.22 s
<code>volta_sgemm_64x64_nt</code>	4.83	9.91 s
<code>softmax_forward_kernel15</code>	2.42	4.96 s
<code>gelu_forward_kernel</code>	1.00	2.05 s

Table 1: Baseline `nvprof` GPU kernel time attribution (top kernels).

Table 1 summarizes GPU kernel time attribution obtained with `nvprof`, while Figure 1 provides a high-level visual breakdown of kernel execution time. The custom kernel `matmul_forward_kernel14` is the dominant hotspot, accounting for 58.65% of total GPU kernel time. The remaining execution time is largely spent in GEMM kernels, indicating that dense linear algebra operations dominate the workload. This observation motivates optimizations targeting matrix multiplications, such as replacing custom kernels with cuBLAS and exploring reduced-precision computation.

As shown in Table 2, `matmul_forward_kernel14` runs for about 2.3 ms per launch and achieves high SM utilization (71%) while using only 42% of the available memory bandwidth, indicating a compute-bound kernel. High register usage (123 registers per thread) and shared-memory usage (32.8 KiB per block) limit achieved occupancy to roughly 45%, leaving little headroom to increase parallelism. With FP32 compute units already heavily utilized and memory not saturated, further FP32-only optimizations are unlikely to deliver substantial speedups, motivating the use of cuBLAS-based implementations.

Metric	Baseline value
Duration	2.28 ms
Compute (SM) Throughput	71.27%
Memory Throughput	41.80%
DRAM Throughput	16.48%
Registers per thread	123
Static shared mem per block	32.77 KiB
Achieved occupancy	45.24%

Table 2: Baseline `ncu` metrics for `matmul_forward_kernel14`.

4.2 Optimization A: cuBLAS linear layers

With cuBLAS linear enabled, the average iteration time was **1118.69 ms/iter**, compared to the baseline **1063.97 ms/iter**, corresponding to a speedup of $S \approx 0.951$ (about a 5.1% slowdown) for this workload. This configuration achieves a throughput of approximately **3660 tokens/s**, slightly lower than the FP32 baseline.

Figure 2 and Table 3 show that kernel attribution shifts from the custom `matmul_forward_kernel14` hotspot to vendor GEMM kernels. In particular, `volta_sgemm_128x64_tn` becomes the dominant kernel (41.81% of GPU time), followed by `volta_sgemm_128x128_tn` (18.80%).

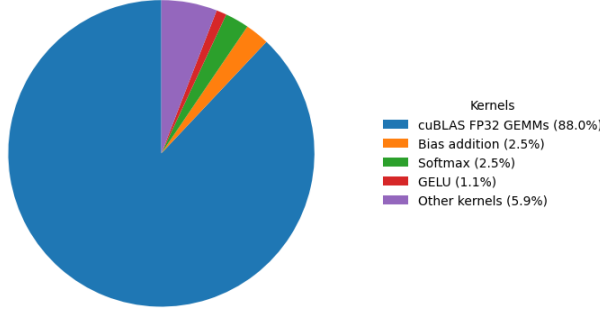


Figure 2: GPU kernel time breakdown with cuBLAS linear enabled (FP32).

Kernel (cuBLAS)	GPU time share [%]	Total time
<code>volta_sgemm_128x64.tn</code>	41.81	81.43 s
<code>volta_sgemm_128x128.tn</code>	18.80	36.61 s
<code>volta_sgemm_128x64.nn</code>	10.22	19.89 s
<code>volta_sgemm_128x64.nt</code>	6.73	13.11 s
<code>volta_sgemm_64x64.nn</code>	5.53	10.76 s
<code>volta_sgemm_64x64.nt</code>	4.87	9.48 s
<code>add.bias_kernel</code>	2.54	4.94 s
<code>softmax_forward_kernel15</code>	2.53	4.92 s

Table 3: nvprof GPU kernel time attribution with cuBLAS linear enabled (top kernels).

Even though the cuBLAS GEMMs now dominate GPU time and Nsight Compute reports solid SM throughput $\approx 72\%$ for the dominant FP32 cuBLAS GEMM kernel (Table 4), the average iteration time didn’t improve. This is due to heavier overhead from kernel launches and still no access to Tensor Cores. Furthermore, they also burn 122 registers/thread and 12.8 KiB of shared memory, so occupancy tops out near 50%: higher than our custom kernel, but not enough to offset the extra per-block work. With bias kernels still negligible, and the overall performance still bounded by the FP32 peak throughput, moving to mixed precision (FP16 GEMMs on Tensor Cores with FP32 master weights) is the natural next optimization.

Metric	Value
Duration	3.03 ms
Compute (SM) Throughput	72.35%
Memory Throughput	48.17%
DRAM Throughput	28.46%
L2 Hit Rate	97.46%
Registers per Thread	122
Static Shared Memory per Block	12.80 KiB
Achieved Occupancy	49.48%

Table 4: NCU metrics for FP32 cuBLAS GEMM (`volta_sgemm_128x64.tn`) without Tensor Core acceleration.

4.3 Optimization B: cuBLAS linear + mixed precision

This section focuses on the cuBLAS and mixed precision configuration. Unlike the FP32 baseline, this configuration is not numerically identical to the reference test outputs: the test reports a logits mismatch and the aggregate gradient tensor check fails under the default absolute tolerance of 10^{-2} . For example, the first reported logit differs by approximately 0.075 (absolute error), exceeding the FP32 test threshold. Such deviations are expected when using FP16 Tensor Core execution and are acceptable in inference scenarios, where exact FP32 reproducibility is not required. We therefore treat this optimization as a reduced-precision variant and evaluate it primarily on throughput, reporting the numerical deviation as a trade-off.

The average iteration time improves from the baseline **1063.97 ms/iter** to **834.44 ms/iter**, corresponding to a speedup of $S \approx 1.275$ (about 27.5% faster). This corresponds to a training throughput of approximately **4910 tokens/s**, compared to **3850 tokens/s** for the FP32 baseline.

Figure 3 and Table 5 show that compute shifts toward tensor-core FP16 GEMMs, while FP32 GEMMs remain present. Mixed precision also adds conversion overhead, with `fp32_to_fp16_kernel` using 3.17% of GPU time.

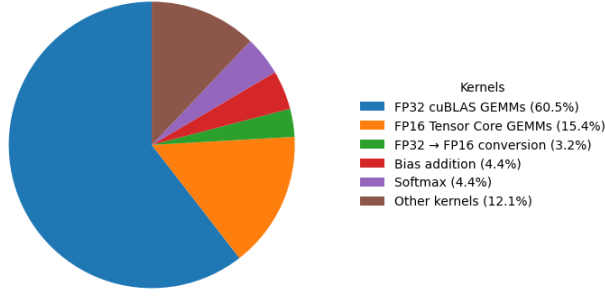


Figure 3: GPU kernel time breakdown with cuBLAS linear and mixed precision enabled.

Kernel (cuBLAS + MP)	GPU time share [%]	Total time
<code>volta_sgemm_128x64_nn</code>	18.66	21.07 s
<code>volta_sgemm_128x64_nt</code>	12.25	13.83 s
<code>volta_sgemm_128x64_tn</code>	11.23	12.69 s
<code>volta_sgemm_64x64_nn</code>	9.63	10.87 s
<code>volta_sgemm_64x64_nt</code>	8.74	9.87 s
<code>turing_s1688gemm_fp16_256x128_ldg8_tn</code>	8.61	9.73 s
<code>turing_s1688gemm_fp16_128x128_ldg8_tn</code>	6.78	7.65 s
<code>softmax_forward_kernel5</code>	4.43	5.00 s
<code>add_bias_kernel</code>	4.39	4.96 s
<code>fp32_to_fp16_kernel</code>	3.17	3.58 s

Table 5: nvprof GPU kernel time attribution with cuBLAS linear and mixed precision enabled (top kernels).

Metric	Value
Duration	1.17 ms
Compute (SM) Throughput	69.06%
Memory Throughput	36.08%
DRAM Throughput	30.52%
Tensor Pipeline Utilization	~72%
L2 Hit Rate	86.40%
Registers per Thread	238
Static Shared Memory per Block	49.15 KiB
Achieved Occupancy	24.95%

Table 6: NCU metrics for the FP16 Tensor Core GEMM kernel (`turing_s1688gemm_fp16_256x128_ldg8_tn`) used by cuBLAS.

Metric	Value
Duration	0.10 ms
Compute (SM) Throughput	25.55%
Memory Throughput	66.20%
DRAM Throughput	66.20%
L1/TEX Hit Rate	16.67%
L2 Hit Rate	33.47%
Registers per Thread	16
Static Shared Memory per Block	0 B
Achieved Occupancy	76.12%

Table 7: NCU metrics for the FP32 to FP16 conversion kernel used in the mixed-precision path.

The FP16 Tensor Core GEMM kernel completes in about 1.17ms per launch while keeping the tensor arithmetic units roughly 72% busy; even though register and shared-memory demands hold occupancy near 25%, Tensor Core execution still improves overall runtime compared to FP32. Table 6 captures the trade-off: modest occupancy but strong tensor utilization and healthy DRAM throughput, matching the observed lifting from 3.85k to 4.91k tokens/s. From Table 7, the conversion kernel’s memory throughput hits 66% of peak while SM throughput is only 26%. Nsight flags it as memory-bound with every launch spending most of its time waiting on memory transfers - long L1/TEX stalls. This explains why small batches/sequence lengths see little speedup — the conversion overhead dominates unless it is fed large matrices.

4.4 Inference-only performance

To better isolate the impact of optimized linear layers and reduced precision, we additionally evaluated inference-only performance by disabling the backward pass and optimizer updates. This configuration reflects a common deployment scenario for LLMs, where forward-pass latency and throughput are the primary performance metrics.

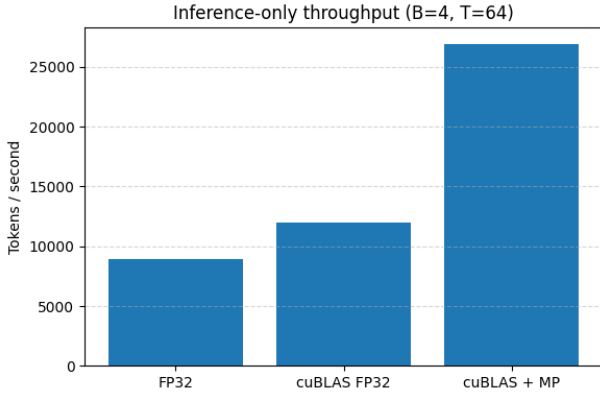


Figure 4: Inference-only throughput comparison.

Configuration	Avg. forward time [ms]	Tokens/s
FP32 baseline	28.79	8,891
cuBLAS FP32	21.43	11,946
cuBLAS + MP	9.50	26,940

Table 8: Inference-only performance summary.

In contrast to the training workload, inference-only execution benefits significantly from optimized GEMM kernels. As seen in Figure 4 and Table 8, using cuBLAS alone improves throughput by about 34%, while combining cuBLAS with mixed precision gives more than a $3\times$ speedup over the FP32 baseline. This highlights that the benefits of cuBLAS and Tensor Cores are most clearly realized in inference-dominated workloads, where large matrix multiplications dominate execution time.

4.5 Inference scalability with sequence length

During inference, B controls how many requests are processed in parallel, while T determines the number of tokens handled per request, together defining the size and cost of each forward pass. Since long-context inference is common in practice, scalability with respect to T is particularly important. To evaluate this, we measured forward-only performance for $T = 64, 256, 1024$ while keeping $B = 4$ fixed.

Seq. length T	Configuration	Avg. latency [ms]	Tokens/s	Speedup
64	FP32 baseline	31.33	8.17k	1.00
64	cuBLAS FP32	18.33	13.97k	1.71
64	cuBLAS + MP	5.54	46.24k	5.66
256	FP32 baseline	75.69	13.53k	1.00
256	cuBLAS FP32	69.16	14.81k	1.09
256	cuBLAS + MP	21.54	47.55k	3.51
1024	FP32 baseline	349.77	11.71k	1.00
1024	cuBLAS FP32	324.27	12.63k	1.08
1024	cuBLAS + MP	130.94	31.28k	2.67

Table 9: Inference performance across sequence lengths. Speedup is computed relative to the FP32 baseline at the same sequence length.

Table 9 shows that the FP32 baseline exhibits limited scalability: while latency increases substantially with sequence length, throughput saturates around 11–13k tokens/s. Enabling cuBLAS for FP32 linear layers improves inference performance, with a significant benefit for short sequences, while giving only modest gains at larger T and not fundamentally changing the overall scaling behavior.

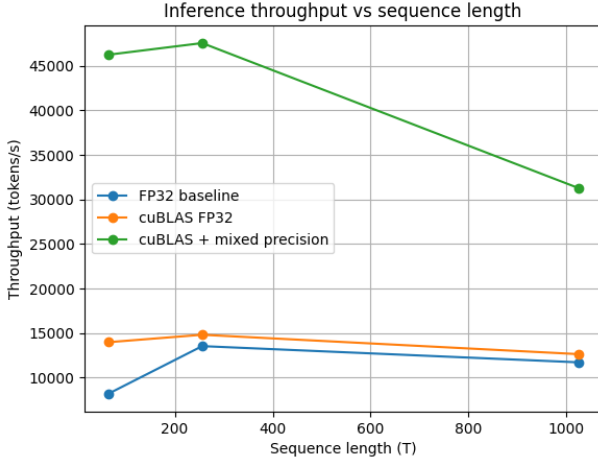


Figure 5: Inference throughput (tokens/s) as a function of sequence length.

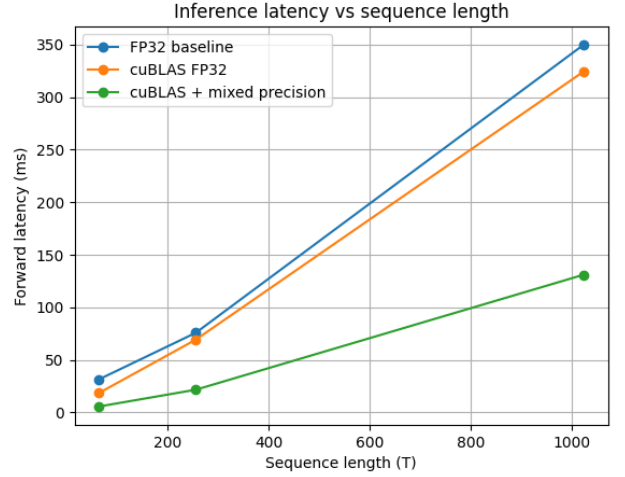


Figure 6: Average forward-pass latency as a function of sequence length.

Figures 5 and 6 provide a clearer visualization of these trends. While FP32 inference shows nearly flat throughput and steadily increasing latency as T grows, mixed-precision inference using cuBLAS scales significantly better. For $T = 64$, throughput improves by more than $5\times$ compared to the FP32 baseline, while for $T = 1024$ a $2.7\times$ speedup is still observed. At the same time, absolute latency is reduced substantially across all sequence lengths.

This behavior reflects improved arithmetic intensity and more effective utilization of Tensor Cores in mixed-precision GEMMs. As the sequence length increases, larger matrix multiplications better amortize kernel launch and conversion overheads, allowing the hardware to operate closer to peak compute efficiency. Overall, these results demonstrate that mixed-precision inference not only reduces latency but also scales significantly better with sequence length, making it particularly well-suited for long-context and high-throughput inference workloads.

5 Conclusion and Future Work

In this project, we accelerated a C/CUDA implementation of a transformer-based LLM by targeting its most expensive operation, `matmul_forward_kernel4`. We replaced the custom FP32 matrix multiplication with cuBLAS and enabled FP16 mixed precision to use Tensor Cores. While cuBLAS alone provided limited benefits for training, the mixed-precision with cuBLAS significantly improved performance, increasing training throughput by about 27% and achieving up to $5\times$ higher inference throughput, with consistent gains for longer sequences.

Although our mixed-precision implementation already accelerates the forward pass, there are several improvements we did not pursue in this project. To begin with, we kept all gradients and optimizer state in FP32, which avoids most stability issues but leaves out additional speed and memory savings. A more aggressive approach would store gradients and optimizer state in FP16 and use dynamic loss scaling to prevent underflow in small gradient values, as is standard in Automatic Mixed Precision (AMP) libraries. Finally, our choice of FP16 was limited by the T4 hardware; on newer architectures that support BF16 Tensor Cores, implementing a BF16 path for both activations and weights would likely offer a better trade-off between dynamic range and performance.

References

- [1] Andrej Karpathy. llm.c: Training gpt-2 in pure c/cuda, 2023. Accessed 2025.
- [2] NVIDIA Corporation. *cuBLAS Library User Guide*, 2024. Accessed 2025.
- [3] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. Nvidia tensor core programmability, performance, and precision. *IEEE Micro*, 38(2):8–16, 2018.
- [4] NVIDIA Corporation. *Mixed Precision Training*, 2023. Accessed 2025.